



Path Planning with Motion Primitives in Dynamic Environments: SIPP on Lattices

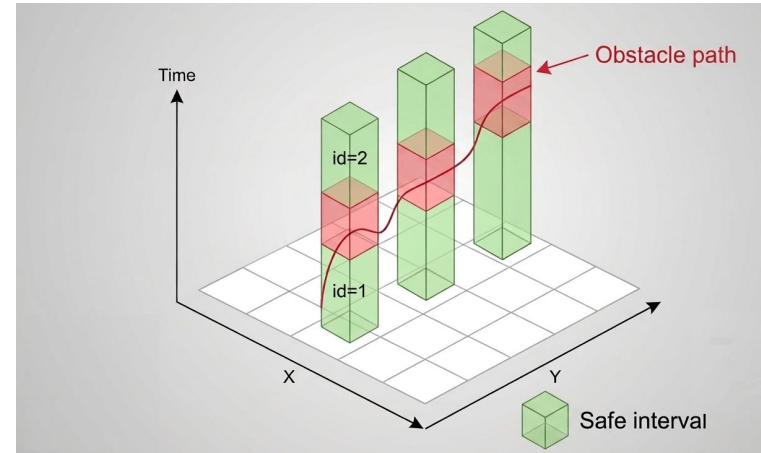
Marat Agranovskiy

Markov Lab, Saint-Petersburg University

18 August 2026, Vladivostok

Motivation: Dynamic Environments

- **The Scenario:** Shared robotic workspaces (e.g., automated warehouses).
- **The Challenge:** Navigating around dynamic obstacles with *known/predictable* trajectories (e.g., other robots).
- **Environment Representation:** Regular grid maps (reflects sensor data).
- **The Baseline Solution:**
Safe Interval Path Planning (SIPP):
 - Condenses continuous time into discrete **Safe Intervals**.
 - Search space of $(x, y, \text{interval_id})$.



Motivation: Lattice-Based Path Planning

Traditional grid planners produce jagged, "stop-and-turn" paths. Geometrically valid, but kinematically harsh for real robots.

Our focus: Lattice-Based Planning:

- Discretizes state space $s = (x, y, \theta)$.
- Motion Primitives: Short, pre-computed, feasible motion segments.
- Algorithm: SIPP searches for a sequence of primitives that smoothly connect.

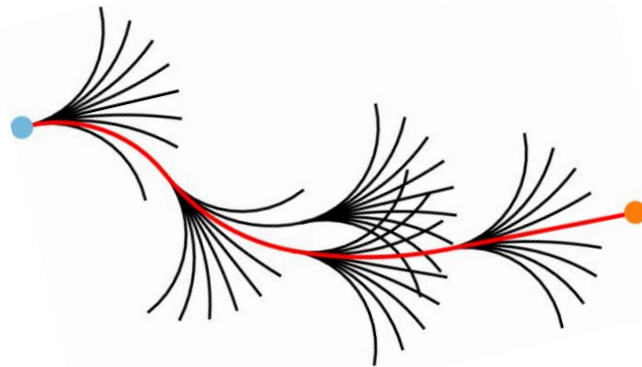
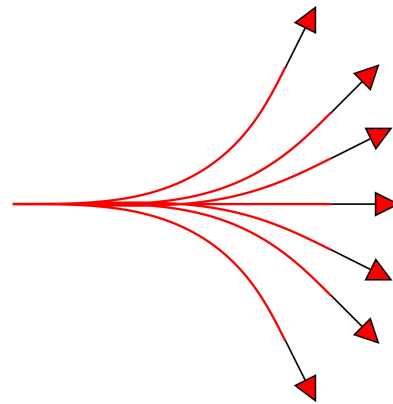


Image from: Thoresen, Marius & Nielsen, Niels & Mathiassen, Kim & Pettersen, K.Y.. (2021). Path Planning for UGVs based on Traversability Hybrid A*. IEEE Robotics and Automation Letters. PP. 1-1. 10.1109/LRA.2021.3056028.

Problem Statement

Dynamic Environment & States

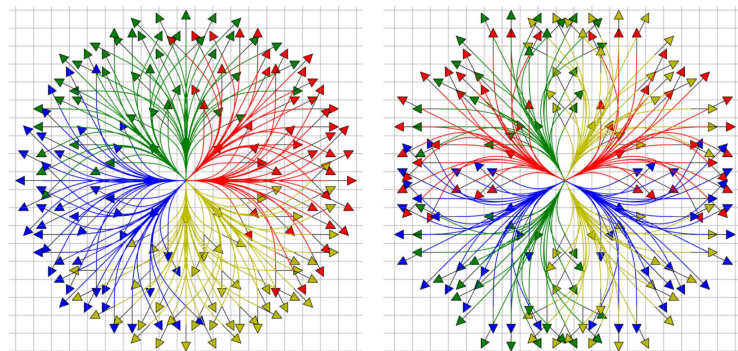
- Discrete state $s = (i, j, \theta)$ on a regular 2D grid.
- Cell $(i, j) \rightarrow$ set of Safe Intervals (periods free from moving obstacles): $D_{ij} = \{ [t_{in}^{(1)}, t_{out}^{(1)}], \dots \}$

Motion Primitives

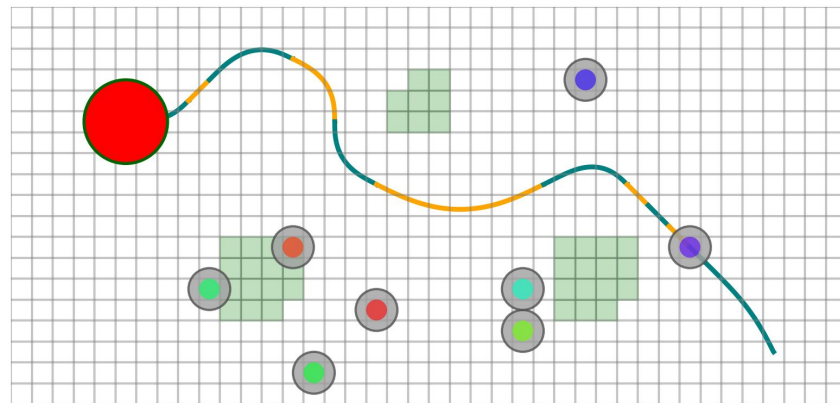
- Short, kinodynamically feasible motions connecting discrete states: $(i, j, \theta) \rightarrow (i', j', \theta')$.
- **Spatio-Temporal Trace:** Primitive sweeps cells c , blocking each during $[T_i \square^c, T_{ou} \square^c)$.

Objective: Find a sequence of collision-free primitives to the goal state minimizing total execution time.

Motion primitives:



total: $24 \times 16 = 384$



Teal/orange primitives avoid moving disks to form a safe path.

Method: SIPP with Motion Primitives

SIPP State:

- Search space is expanded to: $p = \langle i, j, \theta, l \rangle$
- $l = [t_{\square_{ar}}, t_{e\square}]$ – safe interval at cell (i, j) .

Cost Metric (g-value):

- $g(p) = t_{arr}$ (minimum arrival time at this state).

Successor Generation & Waiting:

- Departure Window $[t_{arr}, t_{e\square})$
- Can wait in place for $\tau \geq 0$ before departure.
- Apply primitives strictly from current θ .

Transition:

- Next state $\langle i', j', \theta', l' \rangle$ with a new $t'_{arr} = t_{arr} + \tau + \Delta t$.

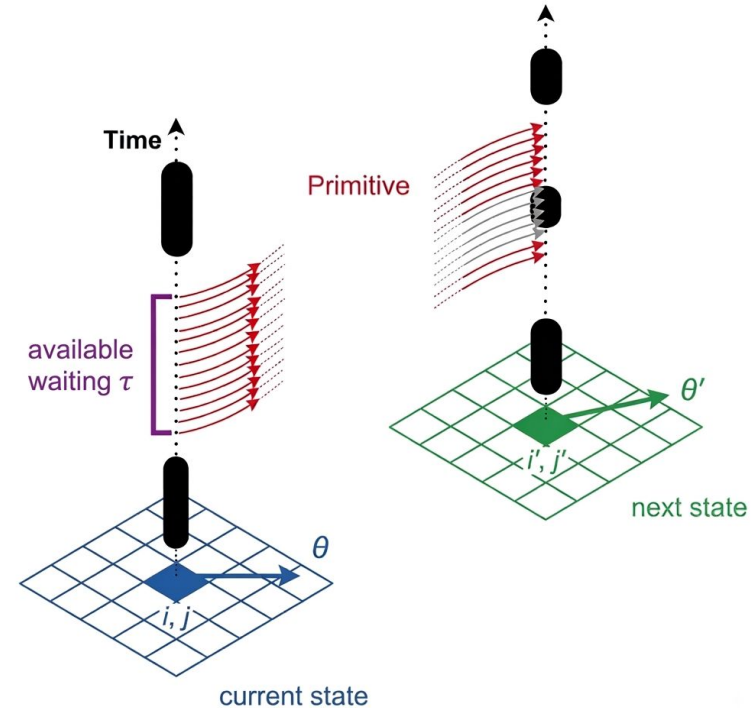


Image inspired by one from:

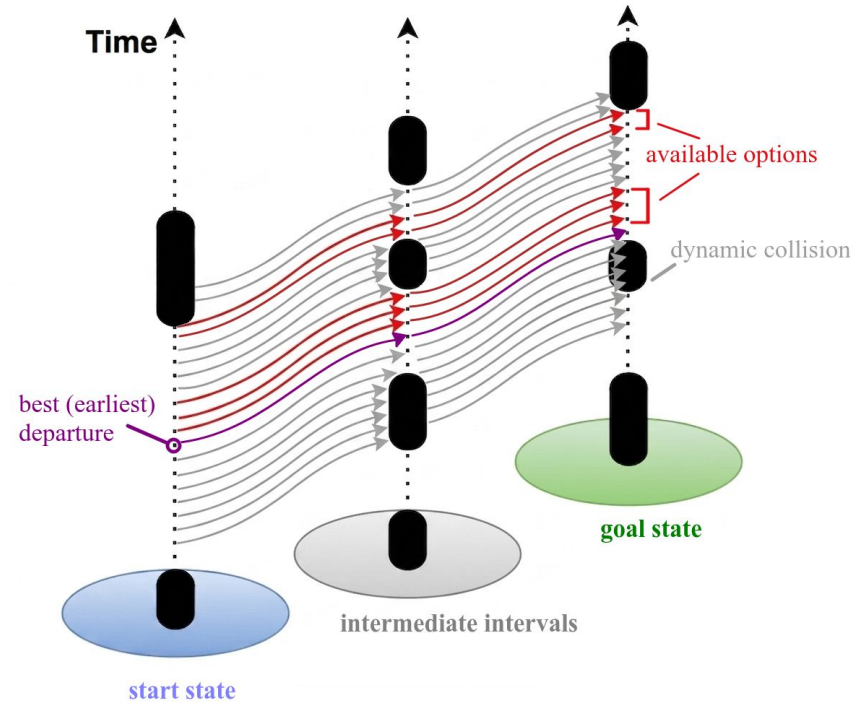
Ali, Z. A., & Yakovlev, K. (2023). Safe Interval Path Planning with Kinodynamic Constraints. Proceedings of the AAAI Conference on Artificial Intelligence, 37(10), 12330–12337. <https://doi.org/10.1609/aaai.v37i10.26453>

Method: Earliest Safe Departure Search

The Challenge: A primitive spans multiple cells, each with its own dynamic obstacles. Checking every possible departure time (sampling) is too slow.

Key Insight: Backward Time Projection

- Project constraints backward to the departure.
- If an obstacle blocks cell at time t , and the primitive reaches one in τ , departure is forbidden at $t - \tau$.
- Constructs "Invalid Departure Blocks".
- Sorts blocks and scans for the first available time gap.

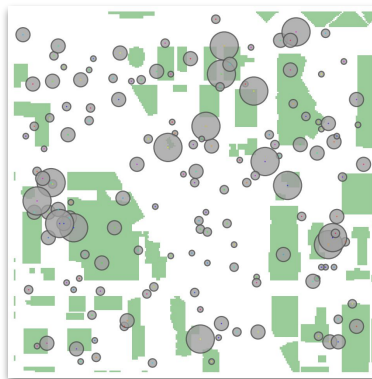


Evaluating a Time-Shifted single primitive against Safe Intervals

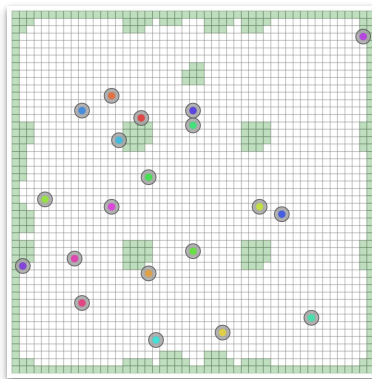
Empirical Evaluation

Setup:

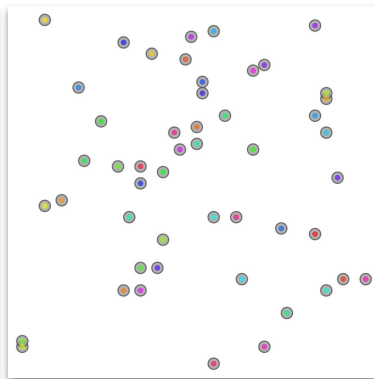
- **Maps:** three MovingAI grids.
- **Instances:** >5,000 start-goal pairs.
- **Control sets:** 2 smooth + 4 for 2^k .



Denver (256x256 grid) with 150 obs.

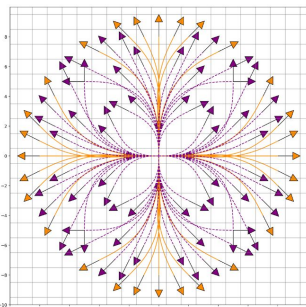


Arena (50x50 grid) with 20 obs.

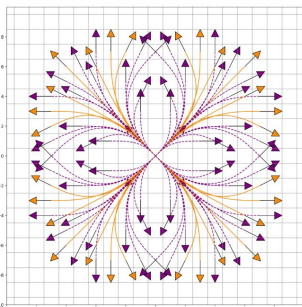


Empty map (64x64) with 50 obs.

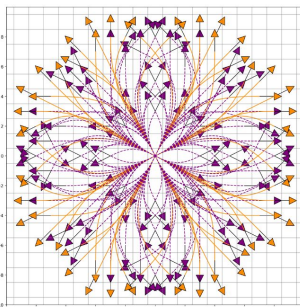
Smooth control sets:



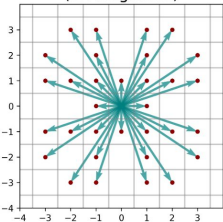
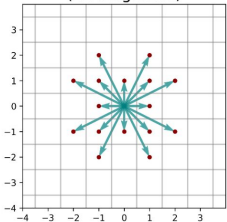
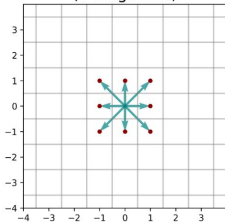
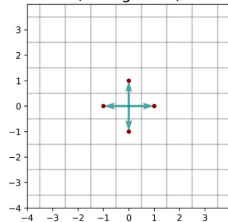
2^2 -connectivity
(4 neighbors)



2^3 -connectivity
(8 neighbors)



2^4 -connectivity
(16 neighbors)

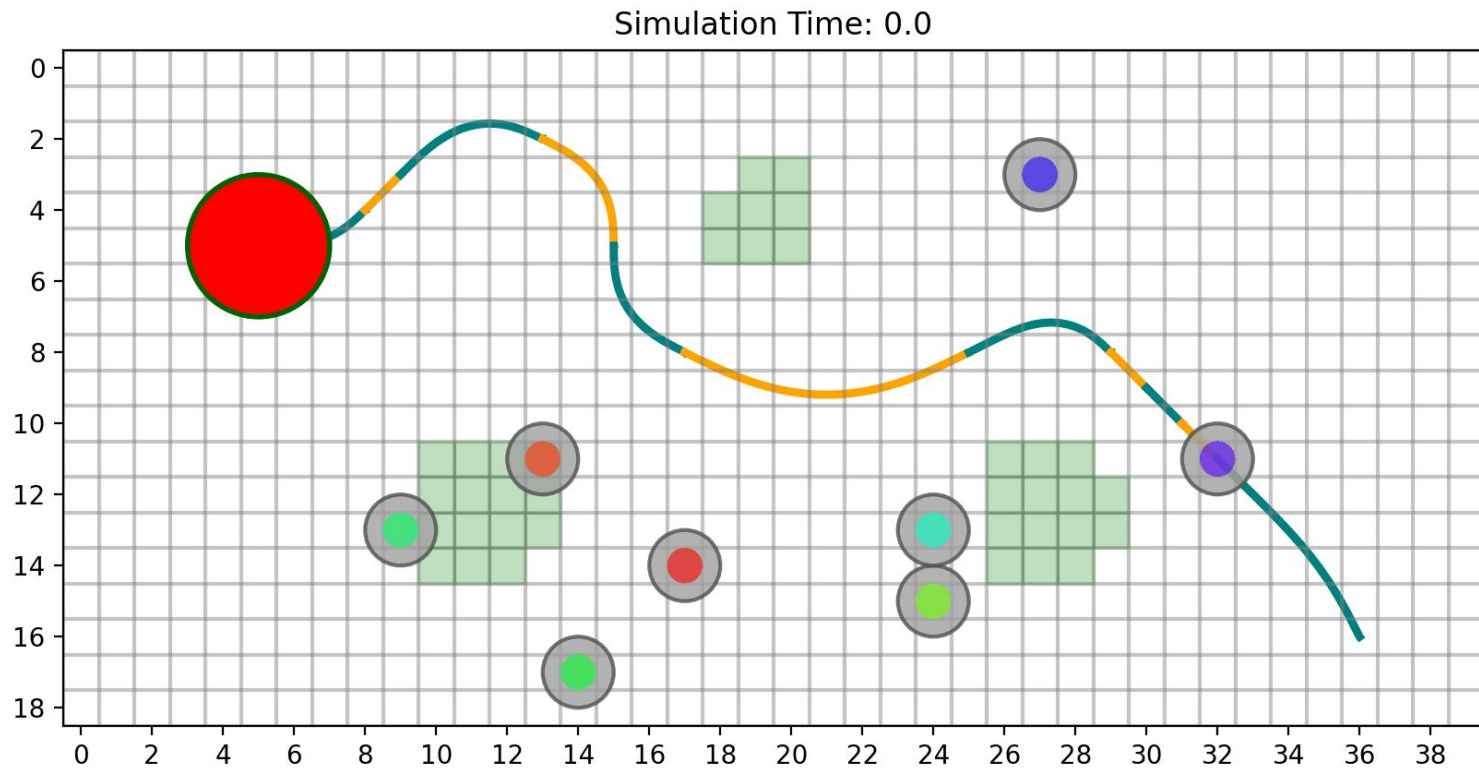


2^5 -connectivity
(32 neighbors)

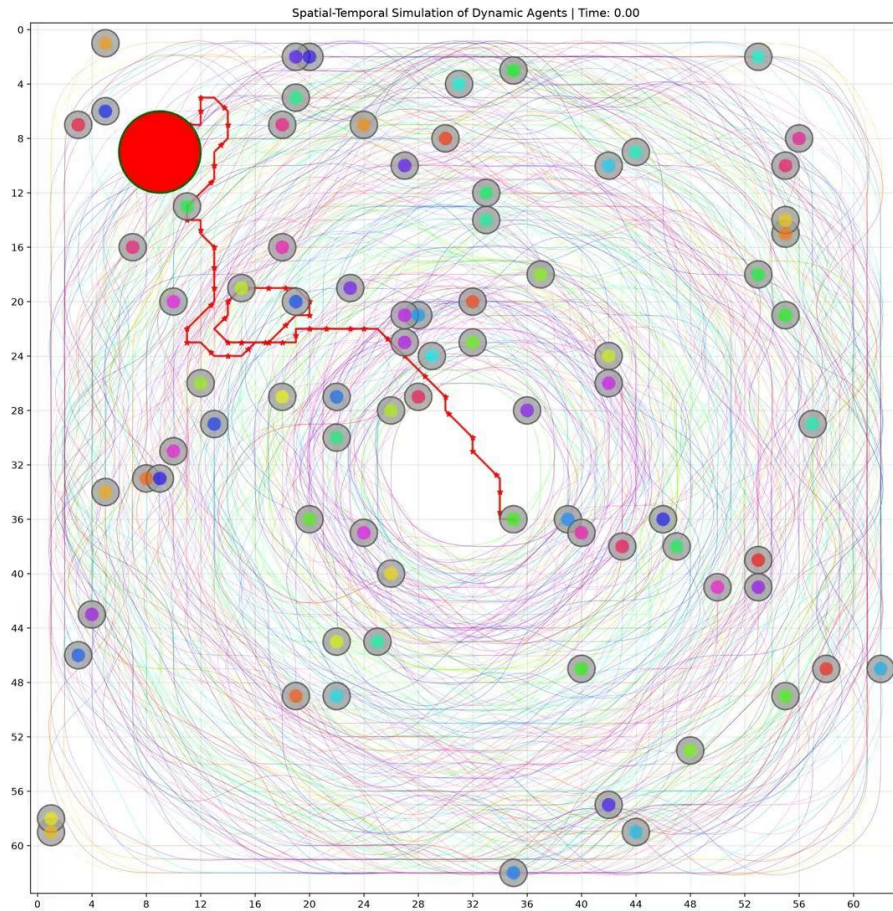
Key results:

- **Computational Cost:** Lattice search is heavier (~20x longer search time) due to 3D state space (with heading).
- **Trajectory Quality:** Lattice paths reduce spatial angularity and bending energy, guaranteeing kinematic feasibility.

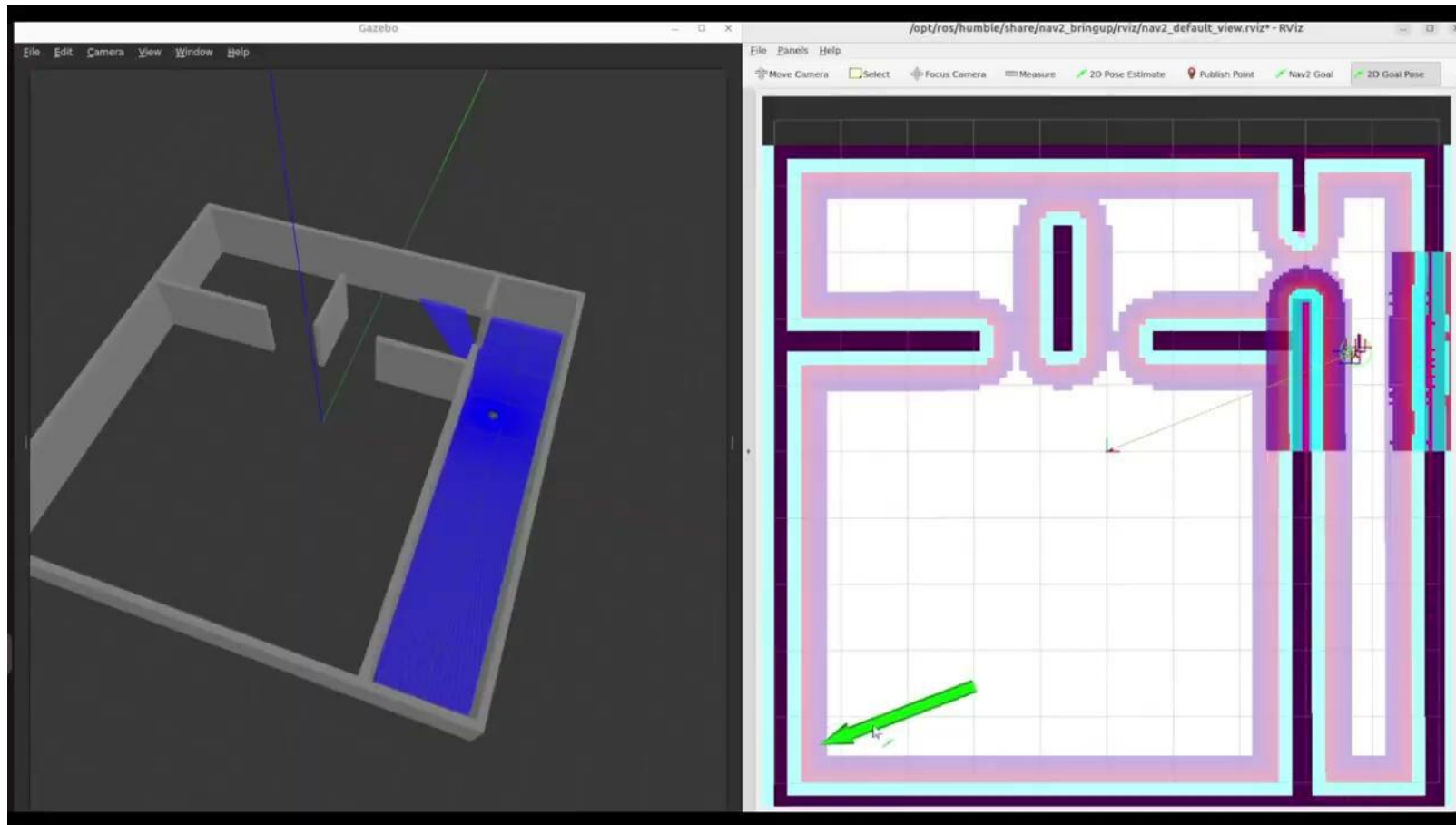
Simulation 1



Simulation 2: hard problem



Simulation 3: ROS2 + Gazebo





Thank you for your attention!

For further questions:

agrinscience@gmail.com



GitHub